

# Written Report

## Reproduction Target

I am reproducing the number of DVWA SQL injection vulnerabilities found by SQLiFuzz with the author's custom crawler.

- Paper citation: I Putu Arya Dharmaadi, Van-Thuan Pham, Fadi Mohsen, and Fatih Turkmen. 2026. SQLiFuzz: Uncovering SQL Injection in Any Web Applications. Proc. ACM Softw. Eng. 3, FSE, Article FSE142 (July 2026). <https://doi.org/10.1145/3808149>
- Page and Table/Figure/Section: Table 4, p. FSE142:14, Section 5.5 ("Result and Discussion"), row A1.
- Metric Label: Number of known DVWA SQL injection vulnerabilities detected by SQLiFuzz using the custom crawler (column "CustCraw" under "SQLiFuzz with"), out of the paper's stated 2 known SQLi cases in DVWA.
- Value the Paper Found: 2 (out of 2 known cases)
- Authors' Artifact URL and Version: Repository URL: <https://github.com/websecfuzz/SQLiFuzz>, SQLiFuzz commit: [1b7e2ede42af8e2b9d6d1c44358851e9d4616db9](https://github.com/websecfuzz/SQLiFuzz/commit/1b7e2ede42af8e2b9d6d1c44358851e9d4616db9)
- Benchmark Slice: Bundled DVWA 2.5, SQLiFuzz commit (functions as checksum since DVWA ships bundled inside): [1b7e2ede42af8e2b9d6d1c44358851e9d4616db9](https://github.com/websecfuzz/SQLiFuzz/commit/1b7e2ede42af8e2b9d6d1c44358851e9d4616db9)
- Configuration: Paper environment: Ubuntu 24.04.3, Python 3.12.3, Docker 28.3.3, My environment: Ubuntu 24.04.5 LTS, Python 3.12.3, Docker 29.8.0, Docker Compose v5.5.1

SQLiFuzz must be run with the custom crawler to get results. There is no output section within the author's GitHub repository that has precomputed output. The result only exists after running SQLiFuzz and obtaining the dvwa-bacfuzz-\*.txt output.

## Metric Semantics

The metric is a count of distinct DVWA endpoints, among the paper's two documented "known SQLi" test cases, where SQLiFuzz's oracle observed a DBMS syntax error caused by a fuzzed parameter value.

The numerator is the number of known-SQLi endpoints where the fuzzer's mutated input caused the DBMS to return a SQL syntax error. Specifically, it is the amount of cases where SQLiFuzz detected a potential SQLi vulnerability.

The denominator is the fixed number of SQLi test cases that the paper's authors manually identified in DVWA (Dharmaadi et al., 2026, Section 5.3.1: "In DVWA and XVWA, we identified two SQL injection test cases per benchmark"). Based on DVWA's own documented vulnerability categories, this denominator corresponds to [/vulnerabilities/sql/](#) and [/vulnerabilities/sqli\\_blind/](#).

- Unit: Count of SQLi cases
- Aggregation: Multiple fuzzed requests can hit the same endpoint with different mutated values (Dharmaadi et al., 2026, Section 3.4.2). The metric deduplicates these down to unique endpoints. This is implemented directly in [scripts/derive\\_a1\\_result.py](#).
- Exclusions: SQLiFuzz explicitly does not count a triggered SQL error as a detection unless the fuzzed value appears in the resulting query. In my own raw output, this exclusion corresponds to the "SQL Error but No Matched Strings" category, which is tracked separately from "Matched SQL Injection Detected" and is not counted toward the derived metric.
- Timeout Policy: SQLiFuzz does not have a timeout policy, however it does have an idle-timeout shutdown mechanism. This mechanism occurs when no new fuzzing activity is observed for a period of time. All three of my runs completed in approximately 20-23 minutes.

The paper's evaluation methodology (Dharmaadi et al., 2026, Section 5.5.4) claims "no significant variance" across repeated runs. My own reproduction directly contradicts this for DVWA specifically: two identical low-security runs, with zero configuration changes between them, produced different known- SQLi detection counts (1/2 vs. 2/2). After investigating the outputs of both runs, it was found that there was a Playwright viewport-visibility failure during the first run. During the crawler's automated form submission on [/vulnerabilities/sqli/](#), the Submit button was outside the visible viewport causing the click to silently fail and preventing any request from being generated. This is nondeterminism in the crawler's DOM-interaction layer, not in SQLiFuzz's detection oracle itself.

One plausible incorrect interpretation of the metric is "The number 2 means SQLiFuzz fully exploited both DVWA SQLi vulnerabilities." The paper states that its oracle only checks for a DBMS syntax error caused by the fuzzed value (Dharmaadi et al., 2026, Section 3.4.3). It does not perform actual exploitation. "Detected" means "confirmed exploitable in principle because the DBMS parser choked on the injected value," not "successfully attacked."

My raw log lines ([\[SQLAnalysis\]](#), [\[HOOK\]](#)) show the oracle checking for a syntax-error response category, not attempting data extraction. I also never observed the tool attempting to

read table contents or bypass authentication. Therefore, SQLiFuzz does not exploit the vulnerability, it just ensures a syntax error is triggered.

## Evidence Chain

- Input/Config → DVWA 2.5 docker container, security level set to “low” (hardcoded in `auto_login/dvwa.py`'s `select_option("low")` call), authenticated as Admin via `login_state/dvwa/Admin.txt` cookie file. SQLiFuzz reverse proxy, DB proxy, and crawler launched together via `scripts/sqlifuzz.sh`
- Executed command → `sudo -E bash -c "source ~/SQLiFuzz-a1/venv/bin/activate && ./sqlifuzz.sh dvwa 8081 /openapi.json bacfuzz"`
- Raw observation (all under `raw/<Run>_Reproduction/`) →
  - `Admin_<timestamp>.txt` - list of crawled DVWA page paths
  - `Admin.json` - session metadata, including confirmation of security=low
  - `bacfuzz_dvwa_<timestamp>.log` - [HOOK], [HTTPRequest], [SQLAnalysis] entries showing per-request analysis and oracle verification
  - `BACFUZZ-<timestamp>.log` - [MAINDRIVER Admin] crawler narration (form detection, filling, button clicks, page navigation)
  - `dvwa-bacfuzz-<timestamp>.txt` - final summary dict and the `###Matched SQL Injection Detected` report block
- Transformation → `scripts/derive_a1_result.py` parses each `dvwa-bacfuzz-*.txt` file, extracts the `###Matched SQL Injection Detected` section (preserved as an intermediate value in `derived/run{N}_*_extracted_section.txt`), then classifies each listed endpoint against the paper's known-SQLi set (`vulnerabilities/sqli/`, `vulnerabilities/sqli_blind/`), deduplicating via a Python set. The per-endpoint classification, with the exact raw line that produced it, is preserved in `derived/run{N}_*_classification_trace.txt`.
- Reported Result → `derived/a1_comparison.csv` — final known-SQLi count per run against the paper's Table 4 A1 value of 2:

| run                                                        | known_sqli_detected | known_sqli_total | matches_paper | additional_endpoints_detected                                     |
|------------------------------------------------------------|---------------------|------------------|---------------|-------------------------------------------------------------------|
| paper_table4_A1                                            | 2                   | 2                | --            |                                                                   |
| dvwa-bacfuzz-emalee-VMware-Virtual-Platform-1789231509.txt | 1                   | 2                | no            | vulnerabilities/brute/                                            |
| dvwa-bacfuzz-emalee-VMware-Virtual-Platform-1789241434.txt | 2                   | 2                | yes           | vulnerabilities/brute/vulnerabilities/csp/vulnerabilities/upload/ |
| dvwa-bacfuzz-emalee-VMware-Virtual-Platform-1789249064.txt | 2                   | 2                | yes           | vulnerabilities/exec/vulnerabilities/upload/                      |

The README provides the basic command for executing the tool but no troubleshooting guidance. During setup, I encountered and had to resolve, none of which are documented in the README:

- Missing `#!/bin/bash` shebang in `sqlifuzz.sh/bacfuzz.sh`, causing the scripts to be misinterpreted by dash on Ubuntu
- Missing version for `LiteLLM` (a dependency in `requirement.txt`)
- sudo's default `PATH` reset causing the pipeline to silently run outside the intended Python venv (missing Playwright/mitmproxy despite both being installed)
- A regression in the repository itself: `general_functions.py` (at HEAD) references `read_cov_from_file`, a function that was removed from the codebase in an earlier commit without updating its caller (`30d8a2df5b7a2503e792f450af75415c66490239`). I restored a stubbed version of this function (not the original, the original's dependencies live in a `utils.py` module that does not exist in any commit of the repository)
- Correspondingly, `crawler/utils.py` does not exist in the repository at all; I reconstructed a minimal version providing only `fuzz_open`, the one function actually required to unblock `AttackSurface.py`'s import chain
- DVWA's `config.inc.php` was reset to its .dist template on every `docker compose up --force-recreate`; I added a bind-mount volumes: entry to `WUT/dvwa/docker-compose.yaml` to persist a working configuration

Everything past this setup, the actual crawl, fuzz, and detection behavior runs using the authors' unmodified logic.

## Environment and Deviations

| Factor       | Paper           | My Environment                                             | Differences & Why Necessary | Direction of Effect               |
|--------------|-----------------|------------------------------------------------------------|-----------------------------|-----------------------------------|
| DVWA version | "2.5" (Table 2) | Bundled copy from SQLiFuzz repo, no version string present | See under the table         | Unresolved by design of benchmark |
| Python       | 3.12.3          | 3.12.3                                                     | None                        | N/A                               |
| mitmproxy    | Not pinned      | 11.0.2                                                     | Fresh Install               | Low Risk                          |
| MySQL        | Not pinned      | 8.4.0                                                      | Bundled w/ DVWA             | Low Risk                          |
| Playwright   | Not pinned      | 1.45.1                                                     | Fresh Install               | Possible - see under table        |

|                |                                                       |                                         |                 |                      |
|----------------|-------------------------------------------------------|-----------------------------------------|-----------------|----------------------|
| OS             | Ubuntu 24.04.3 LTS                                    | Ubuntu 24.04.5 LTS                      | Time elapsed    | Negligible           |
| Docker Engine  | 28.3.3                                                | 29.8.0                                  | Fresh Install   | Low Risk             |
| Docker Compose | Not pinned                                            | v5.5.1                                  | Fresh Install   | Low Risk             |
| CPU/RAM        | Intel Xeon Platinum 8358P, 8 physical cores, 32GB RAM | VMware Workstation VM: 4 vCPUs, 8GB RAM | No cloud access | Possible - see below |

- Playwright version: flagged as the most direct candidate explanation for the run1\_low vs ru2\_low detection variance. Playwright’s auto-scroll-before-click behavior has changed across versions; a version mismatch from the authors’ setup could plausibly affect whether the viewport-visibility check succeeds consistently.
- CPU/RAM: running under a resource-constrained VM is a known difference, necessitated by lack of equivalent hardware access. Whether this contributed to the observed crawler-timing nondeterminism is speculation.
- DVWA version: DVWA does not embed a version string in its source, and its own maintainers state the project does not reliably cut tagged releases, directing users to run “the latest pull of code from GitHub” (digininja/DVWA, GitHub PR #386). This reproduction used the exact DVWA copy bundled inside the SQLiFuzz artifact repo.
- Known effects: the sudo+venv PATH issue, the missing [utils.py/read\\_cov\\_from\\_file](#) regression, the DVWA config-persistence issue, and the Playwright viewport-click failure are all confirmed via direct log/source inspection
- Speculation: the reduced CPU/RAM or virtualization jitter specifically caused the viewport-click race.

## Reproduction Judgment

|                     | Paper (Table 4, A1) | run1_low | run2_low | run3_medium |
|---------------------|---------------------|----------|----------|-------------|
| Known SQLi detected | 2                   | 1        | 2        | 2           |
| Absolute difference | -                   | 1        | 0        | 0           |
| Relative Difference | -                   | 50%      | 0%       | 0%          |

- Classification: Approximately Reproduced

Two of three runs (run2\_low, run3\_medium) matched the paper's reported value exactly (2/2, 0% relative difference). The one run that did not (run1\_low, 50% relative difference) has a fully traced, non-detection-logic root cause: a Playwright viewport-click failure in the crawler that prevented a request from ever being generated for one endpoint, not a failure of SQLiFuzz's SQL injection oracle itself. I classify the reproduction target as approximately reproduced, as the artifact exhibits crawler-level nondeterminism not disclosed in the paper's stated stability claim (Dharmaadi et al., 2026, Section 5.5.4).

## One Debugging Decision

One obstacle I faced during this reproduction was when running the fuzzer it produced an `ImportError: cannot import name 'read_cov_from_file' from 'general_functions'` (Output from error found in [raw/errors/](#)). This blocked the entire crawl/fuzz pipeline from executing.

- First Hypothesis: The code has a typo in [general\\_functions.py](#) or the function has a slightly different naming convention.
- Evidence that Rejected this: When looking at the author's GitHub and observing the [general\\_functions.py](#) code, I found that the [read\\_cov\\_from\\_file](#) function did not exist in the most recent version of the file. Then, I looked to see if there were other commits for this file, and there was one named "first upload". That commit contained the function that was missing. I also found that [crawler/utils.py](#), which is a module that [read\\_cov\\_from\\_file](#) depended on, never existed in the repository.
- What I Changed: I restored a stub version of [read\\_cov\\_from\\_file](#) (returning (0, 0) when no coverage file is found, matching the function's original graceful-fallback behavior, but omitting the calls into the non-existent utils module) and created a minimal [crawler/utils.py](#) providing only [fuzz\\_open](#), the one function actually required elsewhere in the codebase ([AttackSurface.py](#) also imports it directly, independent of the coverage-tracking code path). ([utils.py](#) and [general\\_functions.py](#) found in [/scripts/](#))
- New Evidence that Shows the Change Addressed the Problem: The `ImportError` disappeared and the pipeline did not crash again. The program progressed all the way through crawling ([Admin\\_\\*.txt](#) populated with 45+ discovered links), form-filling ([Identifying 1 form in page log entries](#)), fuzzing (fuzzed\_req: 256+ in the summary dict), and produced SQL injection detections matching known DVWA vulnerabilities ([###Matched SQL Injection Detected](#) entries for [/vulnerabilities/sqli\\_blind/](#) and later [/vulnerabilities/sqli/](#)).

## Prediction-first Variation

I predicted that setting DVWA's security level from low to medium would cause SQLiFuzz to detect 0/2 known DVWA SQLi cases, since medium-level DVWA applies `mysqli_real_escape_string()` to the id parameter, which I expected would prevent apostrophes from being accepted as a method for injection.

- Single Factor Changed: DVWA Security Level (`low` → `medium`), via an edit to `auto_login/dvwa.py`
- Constant Factors: SQLiFuzz commit/reconstruction, `docker-compose.yaml` configuration, host environment, exact invoked command, DVWA version
- Result: SQLiFuzz detected 2/2 known DVWA SQLi cases

After inspecting DVWA's `/var/www/html/vulnerabilities/sqli/source/medium.php` I found that at the medium security level, DVWA is supposed to call `mysqli_real_escape_string`, which escapes quote characters. But when going into `medium.php`, the line `$id = mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $id);` shows that the escape works but since `$id` is not in quotes, it allows the malicious input to still be interpreted as SQL code rather than as user data.

## Assignment 2 Direction

I will continue with this baseline as the reproduction was functional and produced results comparable to the published number. The debugging process has also proven that this project has testable limitations that I am able to investigate further.

One specific boundary/fail-case hypothesis I want to test next is “Does the Playwright viewport-visibility race identified in `run1_low` occur consistently under repeated trials, and can it be eliminated by adding an explicit `scroll_into_view_if_needed()` call before the crawler's button-click step in `main_driver.py/function.py`?

One reason I might change direction is if the viewport-click race turns out to be specific to DVWA. In this case, for Assignment 2, I think I would pivot to testing other crawlers and viewing the results. Specifically, I would like to try Professor Liu's JavaScript recon and crawling approach.

## References

Dharmaadi, I. P. A., Pham, V.-T., Mohsen, F., & Turkmen, F. (2026, June 30). *SQLiFuzz: Uncovering SQL Injection in Any Web Applications*. ACM Digital Library. Retrieved September 13, 2026, from <https://dl.acm.org/doi/pdf/10.1145/3808149>